

Food-Delivery Marketplace

Final Project Report

Course: Database Application and Design

Project: Food-Delivery Marketplace

Team Name: Food Delivery Marketplace Team

Motto: Delivering food, data, and reliability in one platform.

GitHub Repository: <https://github.com/Numusohandsome/food-delivery-marketplace>

Local Development URL: <http://localhost>

Deployed URL: <http://138.68.88.189>

Swagger/OpenAPI: <http://138.68.88.189/docs>

Public API Example: <http://138.68.88.189/api/restaurants>

Grafana: <http://138.68.88.189:3000/login>

Prometheus: <http://138.68.88.189:9090>

Loki Ready: <http://138.68.88.189:3100/ready> **Date:** May 2026

Team Members

Student Name	Student ID	Role
Miraxmedova Samira	U2310169	Frontend + Report
Maxsetov Nurmukhamad	U2310163	Backend Core
Maxsetov Salamat	U2310164	Database + Cache
Mirjalilov Mirkamol	U2310171	Docker Infrastructure + Observability

1. Abstract

This project presents a food-delivery marketplace system developed as a full-stack database application. The platform connects customers, restaurants, couriers, and backend services through a single integrated system. Customers can browse restaurants, open restaurant menus, add food items to a cart, create orders, and track order status updates in real time.

The system was implemented using React and Vite for the frontend, FastAPI for the backend, PostgreSQL as the main relational database, Redis as an additional data store and cache, WebSocket for live order updates, and Docker Compose for container orchestration. Nginx is used as an API Gateway that routes client traffic to the frontend and backend services. The backend is deployed with two replicas to demonstrate gateway routing and load balancing.

The database layer stores the main business entities of the food-delivery domain, including users, restaurants, menu items, orders, order items, payments, couriers, deliveries, order status history, and reviews. The project also includes Alembic migrations, seed data, indexes, query optimization results, and Redis cache design.

The system supports REST API communication for standard operations and WebSocket communication for real-time order status updates. A daily restaurant settlement batch pipeline was designed to calculate restaurant revenue and payouts. In addition, a Token Bucket Rate Limiter was implemented as the from-scratch system component and integrated into the backend.

The full application is containerized and can be started using Docker Compose. The infrastructure includes frontend, backend replicas, PostgreSQL, Redis, Nginx, Prometheus, Grafana, Loki, and Promtail. The final system demonstrates a working customer flow, database-backed application design, real-time communication, containerized infrastructure, observability, and project documentation.

2. Business Scenario and Requirements

2.1 Business Scenario

The selected business scenario is a food-delivery marketplace. The platform allows customers to order food from multiple restaurants through a single web application. Restaurants provide menu items, customers place orders, couriers deliver orders, and the system tracks the complete order lifecycle.

The main goal of the system is to demonstrate a realistic database-driven distributed application. The project combines frontend interaction, backend APIs, relational database design, Redis caching, WebSocket live updates, batch processing, Docker infrastructure, and observability.

The main customer journey is:

- Open marketplace
 - Browse restaurants
 - Select a restaurant
 - View menu
 - Add items to cart
 - Create order
 - Track order status
 - Receive live WebSocket updates

The supported order status flow is:

created → confirmed → preparing → picked_up → delivered

2.2 Actors

Actor	Description
Customer	A user who browses restaurants, selects menu items, creates orders, and tracks order status.
Restaurant Owner	A user responsible for restaurant information, menu items, and order preparation.
Courier	A delivery worker responsible for picking up and delivering customer orders.
Platform Backend	The backend service that manages API requests, order processing, WebSocket updates, rate limiting, and data access.
Administrator / Operator	A person who monitors the system through logs, metrics, dashboards, and documentation.

2.3 Use Cases

Use Case 1: Browse Restaurants

Field	Description
Primary Actor	Customer
Goal	View the list of available restaurants.
Preconditions	The system is running and restaurant data exists.
Primary Flow	The customer opens the frontend application. The frontend sends a request to the restaurant API endpoint. The backend returns restaurant data. The frontend displays restaurant cards.
Result	The customer can select a restaurant and open its menu.

Use Case 2: View Restaurant Menu

Field	Description
Primary Actor	Customer
Goal	View menu items of a selected restaurant.
Preconditions	The customer has selected a restaurant from the restaurant list.
Primary Flow	The customer clicks “View menu”. The frontend sends a request for the selected restaurant menu. The backend returns menu items. The frontend displays item names, descriptions, and prices.
Result	The customer can add menu items to the cart.

Use Case 3: Create Order

Field	Description
Primary Actor	Customer
Goal	Create an order from selected menu items.
Preconditions	The customer has added at least one item to the cart.
Primary Flow	The customer opens the cart page and reviews selected items. The customer clicks “Create order”. The frontend sends an order creation request to the backend. The backend creates the order and returns the order ID and initial status.
Result	The order is created with status <code>created</code> .

Use Case 4: Track Order Status

Field	Description
Primary Actor	Customer
Goal	View the current status of an order.
Preconditions	An order has already been created.
Primary Flow	The customer opens the order status page. The frontend requests order details from the backend. The backend returns the current order status. The frontend displays the order status timeline.
Result	The customer can see the current order status.

Use Case 5: Receive Live Order Updates

Field	Description
Primary Actor	Customer
Goal	Receive real-time order status updates without refreshing the page.
Preconditions	The order status page is open and the backend WebSocket service is running.
Primary Flow	The frontend opens a WebSocket connection for the order. The backend accepts the connection. When the order status changes, the backend sends an update message. The frontend updates the displayed status.
Result	The customer sees live order status changes.

2.4 Functional Requirements

ID	Requirement	Implementation
FR1	The system shall display a list of restaurants.	React restaurant list page and REST API endpoint.
FR2	The system shall display menu items for a selected restaurant.	Menu page and restaurant menu endpoint.
FR3	The system shall allow customers to add menu items to a cart.	Frontend cart page and local cart state.
FR4	The system shall allow customers to create orders.	Cart page and POST order endpoint.
FR5	The system shall display order status.	Order status page and GET order endpoint.
FR6	The system shall support live order status updates.	WebSocket endpoint and frontend WebSocket connection.
FR7	The system shall store structured business data.	PostgreSQL relational schema.
FR8	The system shall use an additional data store.	Redis cache design.
FR9	The system shall support containerized execution.	Docker Compose configuration.
FR10	The system shall include monitoring and logging.	Prometheus, Grafana, Loki, and Promtail.
FR11	The system shall include a from-scratch component.	Token Bucket Rate Limiter.
FR12	The system shall include documentation and diagrams.	Markdown documentation, Mermaid diagrams, screenshots, and final report.

2.5 Non-Functional Requirements

Category	Requirement
Usability	The frontend should provide a simple and understandable customer flow.
Maintainability	The project should be separated into frontend, backend, database, infrastructure, and documentation parts.
Scalability	The backend should support multiple FastAPI replicas behind an Nginx gateway.
Reliability	Docker Compose should start the full system consistently.
Observability	The system should provide metrics, logs, and dashboard screenshots.
Performance	PostgreSQL indexes and Redis caching should improve data access.
Security	Environment variables should be used for secrets and configuration values.
Portability	The project should be runnable on another machine using Docker Compose.

2.6 Scale Assumptions

The project is designed as a course-level prototype of a food-delivery marketplace. The expected scale is moderate and focused on demonstrating system architecture and database design rather than production-level traffic.

Area	Assumption
Restaurants	Tens to hundreds of restaurants.
Menu Items	Hundreds to thousands of menu items.
Orders	Hundreds to thousands of orders in test or demo data.
Concurrent Users	Several simultaneous users during demonstration.
Backend Replicas	Two FastAPI backend replicas behind Nginx.
Database	One PostgreSQL instance for persistent relational data.
Cache	One Redis instance for caching and fast access.

2.7 Latency Targets

Operation	Target
Restaurant list loading	Under 1 second in the local Docker environment.
Menu page loading	Under 1 second in the local Docker environment.
Order creation	Under 2 seconds in the local Docker environment.
Order status request	Under 1 second in the local Docker environment.
WebSocket status update	Near real time after backend status change.

These latency targets are prototype-level targets and are intended to show that the system is responsive enough for a course project demonstration.

2.8 Availability Expectations

The system is expected to run locally and in deployment through Docker Compose. The application should remain available while the required containers are running. The use of two backend replicas improves backend availability compared to a single backend container. If one backend replica becomes unavailable, the gateway can continue routing traffic to the other replica.

The current prototype does not implement full production high availability because PostgreSQL and Redis are single-container services. However, the architecture demonstrates the foundation for future improvements such as managed databases, Redis replication, and production-grade load balancing.

2.9 Security Constraints

Constraint	Description
Environment variables	Database credentials, Redis URL, JWT settings, and service ports are configured through environment variables.
API gateway	Nginx is used as the main public entry point.
Rate limiting	A Token Bucket Rate Limiter is included to reduce excessive request traffic.
Input validation	Backend schemas validate API request and response data.
Secrets handling	Demo secrets are stored in environment configuration and should be replaced before production deployment.
Future improvement	HTTPS, stronger authentication, role-based access control, and secure secret management should be added before production deployment.

3. Domain Model and ER Diagram

3.1 Domain Model Overview

The domain model of the food-delivery marketplace represents the main business objects involved in ordering and delivering food. The system includes customers, restaurants, menu items, orders, payments, couriers, deliveries, status history, and reviews.

The database was designed around the main user journey: a customer selects a restaurant, chooses menu items, creates an order, pays for the order, and tracks delivery status. Restaurants manage menu items, couriers handle deliveries, and the system stores status changes for audit and tracking purposes.

The main domain entities are:

Entity	Description
users	Stores customer, restaurant owner, courier, and admin account data.
user_addresses	Stores delivery addresses connected to users.
restaurant_categories	Stores restaurant category data.
restaurants	Stores restaurant information and links restaurants to owners and categories.
menu_items	Stores menu items, descriptions, prices, and availability.
orders	Stores customer orders and order status.
order_items	Stores individual menu items inside an order.
payments	Stores payment method and payment status for an order.
couriers	Stores courier profile and availability status.
deliveries	Stores delivery assignment and delivery progress.
order_status_history	Stores historical order status changes.
reviews	Stores customer reviews and ratings for completed orders.

3.2 Main Relationships

The schema uses one-to-many and one-to-one relationships to represent the food-delivery business process.

Relationship	Type	Description
users → user_addresses	One-to-many	One user can have multiple saved delivery addresses.
users → restaurants	One-to-many	One restaurant owner can manage multiple restaurants.
restaurant_categories → restaurants	One-to-many	One category can include multiple restaurants.
restaurants → menu_items	One-to-many	One restaurant can have many menu items.
users → orders	One-to-many	One customer can create many orders.
restaurants → orders	One-to-many	One restaurant can receive many orders.
orders → order_items	One-to-many	One order can contain many menu items.
menu_items → order_items	One-to-many	One menu item can appear in many order items.
orders → payments	One-to-one	One order has one payment record.
orders → deliveries	One-to-one	One order has one delivery record.
couriers → deliveries	One-to-many	One courier can handle many deliveries over time.
orders → order_status_history	One-to-many	One order can have multiple status history records.
orders → reviews	One-to-one	One completed order can have one review.

3.3 ER Diagram

The ER diagram below summarizes the main database entities and relationships.

erDiagram

```
users ||--o{ user_addresses : has
users ||--o{ restaurants : owns
users ||--o{ orders : places
users ||--o{ reviews : writes
```

```
restaurant_categories ||--o{ restaurants : categorizes
```

```
restaurants ||--o{ menu_items : offers
```

```
restaurants ||--o{ orders : receives
```

```
restaurants ||--o{ reviews : receives
```

```
orders ||--o{ order_items : contains
```

```
menu_items ||--o{ order_items : included_in
```

```
orders ||--|| payments : has
```

```
orders ||--|| deliveries : has
```

```
orders ||--o{ order_status_history : tracks
```

```
orders ||--o| reviews : reviewed_by
```

```
couriers ||--o{ deliveries : handles
```

```
users {
```

```
    int id PK
```

```
    string full_name
```

```
    string email
```

```
    string password_hash
```

```
    string role
```

```
    datetime created_at
```

```
}
```

```
user_addresses {
```

```
    int id PK
```

```
    int user_id FK
```

```
    string address_line
```

```
    string city
```

```
    string phone_number
```

```
}
```

```
restaurant_categories {
```

```
    int id PK
```

```
    string name
```

```
}
```

```
restaurants {
  int id PK
  int owner_id FK
  int category_id FK
  string name
  string description
  string address
  boolean is_active
}

menu_items {
  int id PK
  int restaurant_id FK
  string name
  string description
  decimal price
  boolean is_available
}

orders {
  int id PK
  int user_id FK
  int restaurant_id FK
  string status
  decimal total_price
  datetime created_at
}

order_items {
  int id PK
  int order_id FK
  int menu_item_id FK
  int quantity
  decimal unit_price
}

payments {
```



```
    int id PK
    int order_id FK
    string payment_method
    string payment_status
    decimal amount
}

couriers {
    int id PK
    int user_id FK
    string status
}

deliveries {
    int id PK
    int order_id FK
    int courier_id FK
    string delivery_status
    datetime picked_up_at
    datetime delivered_at
}

order_status_history {
    int id PK
    int order_id FK
    string old_status
    string new_status
    datetime changed_at
}

reviews {
    int id PK
    int order_id FK
    int user_id FK
    int restaurant_id FK
    int rating
}
```

```
    string comment
}
```

3.4 Relational Schema Summary

The relational schema was implemented in PostgreSQL. The schema focuses on data integrity, normalized structure, and support for the main application workflows.

The most important database tables are:

Table	Purpose
users	Stores platform users and roles.
restaurants	Stores restaurants connected to owners and categories.
menu_items	Stores available food items for each restaurant.
orders	Stores customer orders and order status.
order_items	Stores items selected inside each order.
payments	Stores payment information for orders.
couriers	Stores courier profiles and availability.
deliveries	Stores delivery assignment and delivery lifecycle.
order_status_history	Stores order status changes for tracking and audit.
reviews	Stores customer feedback and ratings.

Primary keys are used to uniquely identify records. Foreign keys are used to enforce relationships between users, restaurants, orders, menu items, payments, deliveries, and reviews. Check constraints are used for controlled values such as user roles, order statuses, payment statuses, delivery statuses, courier statuses, quantities, prices, and ratings.

3.5 Data Integrity Rules

Data integrity is enforced using the following mechanisms:

Mechanism	Usage
Primary keys	Each table has a unique identifier.
Foreign keys	Relationships between tables are enforced at the database level.
Unique constraints	Values such as user email and category names are kept unique where required.
Check constraints	Controlled fields such as status, role, price, quantity, and rating are validated.
Not-null constraints	Required fields cannot be empty.

Examples of constrained fields include:

Field	Constraint Purpose
users.role	Allows only valid user roles such as customer, owner, courier, or admin.
orders.status	Allows only valid order statuses.
payments.payment_status	Allows only valid payment states.
menu_items.price	Prevents negative prices.
order_items.quantity	Prevents zero or negative quantities.
reviews.rating	Restricts ratings to the accepted range.

3.6 Database Migrations and Seed Data

Alembic was used for database migrations. The migration system allows the schema to be versioned and recreated consistently in different environments. The project includes migrations for creating the core food-delivery tables, adding performance indexes, and inserting initial seed data.

Seed data was added to make the system testable during development and demonstration. The seed data includes sample users, restaurants, categories, menu items, orders, and related records. This allowed the frontend and backend teams to test the main user flow without manually inserting data each time.

3.7 Indexing and Optimization

Indexes were added to improve query performance for frequently accessed data. The most important query paths include restaurant search, restaurant menu loading, user order history, and order status tracking.

Examples of optimization targets:

Query Area	Optimization Goal
Restaurant list	Faster loading of active restaurants.
Restaurant menu	Faster lookup of menu items by restaurant.
User orders	Faster retrieval of orders by user and creation time.
Order status	Faster lookup of order status and history.
Restaurant search	Faster text search by restaurant name.

These optimizations support the frontend flow and improve the responsiveness of the application during testing and demonstration.

4. System Architecture

4.1 Architecture Overview

The system architecture of the food-delivery marketplace follows a containerized full-stack design. The frontend, backend, database, cache, gateway, and observability services run as separate Docker containers. This separation makes the system easier to develop, test, deploy, and monitor.

The user accesses the application through the browser. The browser sends requests to the Nginx API Gateway. Nginx works as the single public entry point and routes traffic to the React frontend and FastAPI backend replicas. The backend communicates with PostgreSQL for persistent data storage and Redis for cache-related operations.

The architecture also includes WebSocket support for real-time order status updates. When a customer opens the order status page, the frontend establishes a WebSocket connection with the backend through the gateway. When the order status changes, the backend sends an update to the frontend without requiring a page refresh.

The observability layer includes Prometheus for metrics collection, Grafana for dashboards, Loki for logs, and Promtail for log forwarding.

4.2 High-Level System Architecture Diagram

flowchart LR

User[Customer Browser] --> Frontend[React + Vite Frontend]

Frontend --> Gateway[Nginx API Gateway]

Gateway --> Backend1[FastAPI Backend Replica 1]

Gateway --> Backend2[FastAPI Backend Replica 2]

Backend1 --> PostgreSQL[(PostgreSQL Database)]

Backend2 --> PostgreSQL

```
Backend1 --> Redis[(Redis Cache)]
Backend2 --> Redis

Backend1 --> WebSocket[WebSocket Order Updates]
Backend2 --> WebSocket

Backend1 --> RateLimiter[Token Bucket Rate Limiter]
Backend2 --> RateLimiter

Backend1 --> BatchPipeline[Daily Settlement Batch Pipeline]
Backend2 --> BatchPipeline

BatchPipeline --> PostgreSQL

Prometheus[Prometheus] --> Backend1
Prometheus --> Backend2

Grafana[Grafana Dashboard] --> Prometheus

Promtail[Promtail] --> Loki[Loki Logs]
Loki --> Grafana
```

The diagram shows the main runtime architecture. The frontend communicates through the gateway rather than directly accessing backend containers. The gateway routes REST and WebSocket traffic to the backend replicas. PostgreSQL stores persistent relational data, and Redis supports additional fast data access. Observability components monitor and log system behavior.

4.3 Main Components

Component	Technology	Responsibility
Frontend	React + Vite	Provides user interface for restaurant browsing, menu viewing, cart, order creation, and order status tracking.
Backend	FastAPI	Provides REST APIs, WebSocket endpoint, business logic, rate limiting, and database access.
Database	PostgreSQL	Stores persistent relational business data.
Cache	Redis	Provides additional data store and cache design for frequently accessed data.
Gateway	Nginx	Routes public traffic to frontend and backend services.
Container Orchestration	Docker Compose	Runs the full multi-container system.
Metrics	Prometheus	Collects service metrics.
Dashboard	Grafana	Visualizes metrics and logs.
Logs	Loki + Promtail	Collects and queries application/container logs.

4.4 Request Flow

The typical request flow is:

- User Browser
- Nginx Gateway
- FastAPI Backend Replica
- PostgreSQL / Redis
- Backend Response
- Nginx Gateway
- Frontend

For example, when a customer opens the restaurant list page, the frontend calls the restaurant API endpoint through Nginx. Nginx forwards the request to one of the backend replicas. The backend retrieves data from the database or mock data layer and returns the response to the frontend.

4.5 WebSocket Flow

The WebSocket flow is used for live order status updates:

- Order Status Page
- WebSocket connection through Nginx
- Backend WebSocket endpoint
- Status update event
- Frontend updates status in real time

The WebSocket endpoint follows this pattern:

```
/ws/orders/{order_id}
```

The frontend displays the WebSocket connection state on the order status page. During final testing, the page showed:

```
WebSocket: connected
```

This confirms that the real-time update channel was working through the Docker Compose and Nginx setup.

4.6 Deployment-Oriented Structure

The system is structured to be deployed as a set of services instead of one monolithic process. Each service has a focused responsibility. This makes the project closer to a real-world deployment model.

The main deployment characteristics are:

Feature	Implementation
Single public entry point	Nginx gateway on port 80.
Backend replication	Two FastAPI backend containers.
Persistent database	PostgreSQL container with named volume.
Cache/additional store	Redis container.
Observability stack	Prometheus, Grafana, Loki, and Promtail.
Local deployment	Docker Compose.
Configuration	Environment variables and <code>.env</code> file.

4.7 Architecture Limitations

The architecture is suitable for a course-level prototype, but it has some limitations:

Limitation	Explanation
Single PostgreSQL instance	The database is not replicated in the prototype.
Single Redis instance	Redis is not configured as a cluster.
Demo secrets	Environment variables use demo values and should be replaced in production.
Local deployment focus	The project is mainly tested through local Docker Compose.
Limited production hardening	HTTPS, advanced authentication, and production-grade secret management are future improvements.

Despite these limitations, the architecture demonstrates the required full-stack system design, API gateway usage, backend replicas, real-time communication, database integration, and observability.

5. API Design

5.1 API Overview

The backend exposes REST API endpoints for the main food-delivery marketplace operations. The frontend uses these endpoints to load restaurants, display restaurant menus, create orders, retrieve order status, and update order status during the demonstration.

The system also includes a WebSocket endpoint for live order status updates. REST is used for request-response operations, while WebSocket is used for real-time communication.

The API is accessed through the Nginx API Gateway. In the Docker Compose environment, the frontend uses the gateway URLs:

```
REST API base URL: http://localhost/api  
WebSocket base URL: ws://localhost/ws
```

5.2 REST API Endpoints

Method	Endpoint	Purpose
GET	/api/restaurants	Returns the list of available restaurants.
GET	/api/restaurants/{restaurant_id}/menu	Returns menu items for a selected restaurant.
POST	/api/orders	Creates a new customer order.
GET	/api/orders/{order_id}	Returns order details and current status.
PATCH	/api/orders/{order_id}/status	Updates the status of an order.
GET	/api/couriers	Returns courier-related data.
POST	/api/auth/register	Registers a user account.
POST	/api/auth/login	Authenticates a user.

The frontend mainly uses the restaurant, menu, order creation, order retrieval, and order status update endpoints.

5.3 Restaurant List Endpoint

Request

GET /api/restaurants

Response Example

```
[
  {
    "id": 1,
```

```
"name": "Pizza House",
"description": "Italian pizza, pasta and drinks",
"address": "Main Street 12",
"rating": 4.6
}
]
```

Usage

This endpoint is used by the restaurant list page. The frontend calls the endpoint when the user opens the marketplace home page.

5.4 Restaurant Menu Endpoint

Request

GET /api/restaurants/{restaurant_id}/menu

Response Example

```
[
  {
    "id": 101,
    "restaurant_id": 1,
    "name": "Margherita Pizza",
    "description": "Classic pizza with tomato and cheese",
    "price": 8.99
  }
]
```

Usage

This endpoint is used by the menu page. When the customer clicks “View menu”, the frontend requests menu items for the selected restaurant.

5.5 Order Creation Endpoint

Request

POST /api/orders

Request Body Example

```
{
  "restaurant_id": 1,
  "customer_id": 1,
  "delivery_address": "Demo address, Tashkent",
  "total_price": 19.98,
  "items": [
    {
      "menu_item_id": 101,
      "quantity": 1
    },
    {
      "menu_item_id": 102,
      "quantity": 1
    }
  ]
}
```

Response Example

```
{
  "id": 1001,
  "status": "created"
}
```

Usage

This endpoint is called from the cart page when the customer clicks “Create order”. After the order is created, the frontend redirects the user to the order status page.

5.6 Order Status Endpoint

Request

```
GET /api/orders/{order_id}
```

Response Example

```
{
  "id": 1001,
  "status": "created",
  "total_price": 19.98,
  "items": [
    {
      "menu_item_id": 101,
      "quantity": 1
    }
  ]
}
```

Usage

This endpoint is used by the order status page. The frontend loads the current order details before opening the WebSocket connection.

5.7 Order Status Update Endpoint

Request

```
PATCH /api/orders/{order_id}/status
```

Request Body Example

```
{  
  "status": "confirmed"  
}
```

Response Example

```
{  
  "id": 1001,  
  "status": "confirmed"  
}
```

Usage

This endpoint is used during demonstration to move an order through the status lifecycle:

```
created → confirmed → preparing → picked_up → delivered
```

5.8 WebSocket Endpoint

The WebSocket endpoint is used for live order status updates.

Endpoint

```
/ws/orders/{order_id}
```

Message Example

```
{  
  "order_id": 1001,  
  "status": "preparing"  
}
```

Frontend Behavior

When the order status page opens, the frontend connects to the WebSocket endpoint for that order. If the connection is successful, the page shows:

```
WebSocket: connected
```

When the backend sends a status update message, the frontend updates the displayed status without refreshing the page.

5.9 API Design Notes

The API design separates standard request-response operations from real-time updates:

Communication Type	Used For
REST API	Restaurants, menus, order creation, order retrieval, order status update.
WebSocket	Live order status updates.

This design keeps the API simple while still demonstrating an additional API style for real-time communication.

5.10 API Limitations

The current API implementation is suitable for a course project prototype. However, several improvements would be needed for a production system:

Limitation	Future Improvement
Demo authentication	Add full role-based authentication and authorization.
Limited validation in prototype flow	Add stronger validation for all request payloads.
Simple order status update logic	Add role-based restrictions for who can update order status.
Local Docker URLs	Replace local URLs with production domain and HTTPS.
Prototype WebSocket behavior	Add reconnection strategy and persistent event delivery.

6. Data-Layer Design

6.1 Data-Layer Overview

The data layer of the food-delivery marketplace is implemented using PostgreSQL, SQLAlchemy, Alembic migrations, and Redis cache design. PostgreSQL is used as the primary relational database because the project requires structured data,

relationships, constraints, transactions, and query optimization.

The data layer supports the main business workflow of the system:

- Customer
 - Restaurant
 - Menu Items
 - Order
 - Order Items
 - Payment
 - Delivery
 - Status History
 - Review

The database schema was designed to represent the main entities of the food-delivery marketplace and to maintain referential integrity between them.

6.2 PostgreSQL Schema

The relational schema includes the following main tables:

Table	Description
users	Stores customers, restaurant owners, couriers, and admins.
user_addresses	Stores saved delivery addresses for users.
restaurant_categories	Stores restaurant category names.
restaurants	Stores restaurants, owners, categories, and restaurant details.
menu_items	Stores food items offered by restaurants.
orders	Stores customer orders and current order status.
order_items	Stores individual items inside an order.
payments	Stores payment information for each order.
couriers	Stores courier profile and availability status.
deliveries	Stores courier assignment and delivery status.
order_status_history	Stores changes in order status over time.
reviews	Stores customer ratings and comments.

The schema is normalized to avoid unnecessary duplication. For example, order-level data is stored in the `orders` table, while individual food items inside an order are stored in the `order_items` table.

6.3 SQLAlchemy Models

SQLAlchemy models are used to represent database tables in the backend application. Each model corresponds to a relational table and defines columns, data types, relationships, and constraints.

The backend uses SQLAlchemy models for:

Model Area	Purpose
User models	Represent platform users and roles.
Restaurant models	Represent restaurants and menu items.
Order models	Represent orders and order items.
Courier models	Represent courier and delivery data.
Payment models	Represent payment records and statuses.
Review models	Represent customer feedback.

Using ORM models allows the backend to interact with database records through Python classes while still preserving relational database structure.

6.4 Alembic Migrations

Alembic is used for database schema migrations. Migrations make the database structure version-controlled and reproducible across different environments.

The project includes migrations for:

Migration Purpose	Description
Core schema creation	Creates the main food-delivery marketplace tables.
Backend core tables	Adds backend-specific schema structure.
Performance indexes	Adds indexes for frequently queried fields.
Seed data	Inserts initial demo data for testing.

This approach allows the team to recreate and update the database schema consistently using migration commands.

6.5 Seed Data

Seed data was added to support development and demonstration. It allows the system to show meaningful data immediately after setup.

Seed data includes:

Seed Data Type	Purpose
Users	Demo customers, owners, couriers, and admins.
Restaurants	Sample restaurants for the marketplace.
Categories	Restaurant categories.
Menu items	Food items connected to restaurants.
Orders	Example orders for testing status flow.
Related records	Payments, deliveries, and status history where applicable.

Seed data helped the frontend and backend teams test the application without manually inserting records every time.

6.6 Constraints and Validation

The database uses constraints to enforce data correctness.

Constraint Type	Purpose
Primary key	Uniquely identifies each record.
Foreign key	Enforces relationships between tables.
Unique constraint	Prevents duplicate values where required.
Check constraint	Restricts fields to valid values.
Not-null constraint	Ensures required fields are provided.

Examples of validation rules:

Field	Rule
users.email	Must be unique.
users.role	Must be one of the allowed user roles.
orders.status	Must follow allowed order statuses.
payments.payment_status	Must use valid payment status values.
menu_items.price	Must be greater than or equal to zero.
order_items.quantity	Must be greater than zero.
reviews.rating	Must be within the accepted rating range.

These rules reduce invalid data and improve system reliability.

6.7 Indexing Strategy

Indexes were added to improve performance for common queries. The main goal of indexing is to speed up data retrieval for the most frequently used application operations.

Important query patterns include:

Query Pattern	Reason
Get restaurants by active status and category	Used by restaurant browsing.
Get menu items by restaurant ID	Used by menu page loading.
Get orders by user ID and creation time	Used for customer order history.
Get orders by restaurant ID and status	Used for restaurant order management.
Get order status history by order ID	Used for tracking and audit.
Search restaurants by name	Used for restaurant search optimization.

Indexes improve response time, especially as the amount of data grows.

6.8 Redis Cache Design

Redis is used as an implemented additional data store and cache layer. The purpose of Redis is to reduce repeated database reads for frequently accessed data and to support short-lived state.

In the current implementation, Redis caching is used for the restaurant list and restaurant menu endpoints. The backend first checks Redis before reading from PostgreSQL. If cached data exists, the backend returns the cached JSON response. If the cache is empty, the backend queries PostgreSQL and stores the result in Redis with a time-to-live value.

Implemented cache targets include:

Cache Target	Reason
Restaurant list	Frequently loaded by customers.
Restaurant menu	Frequently loaded after selecting a restaurant.
Order status	Frequently checked by the order status page.
Rate limiter counters	Useful for request limiting logic.

Example Redis key patterns:

Key Pattern	Purpose
restaurants:list	Cached list of active restaurants.
restaurants:{restaurant_id}:menu	Cached menu items for a selected restaurant.
restaurants:active	Cache active restaurant list.
restaurant:{id}:menu	Cache menu items for a restaurant.
order:{id}:status	Cache current order status.
rate_limit:{client_id}	Store rate limiter token/counter state.

Redis improves responsiveness and reduces unnecessary load on PostgreSQL.

Despite these limitations, the data layer demonstrates relational database design, schema migration, seed data, constraints, indexes, and implemented Redis caching for frequently accessed restaurant and menu data.

6.9 Data Access Flow

A typical data access flow is:


```
Frontend request
→ Nginx Gateway
→ FastAPI Backend
→ SQLAlchemy Session
→ PostgreSQL
→ Backend Response
→ Frontend
```

When caching is used, the flow can be:

```
Frontend request
→ Nginx Gateway
→ FastAPI Backend
→ Redis cache check
→ Return cached data if available
→ Otherwise query PostgreSQL
→ Store result in Redis
→ Return response
```

This design allows the system to combine reliable relational storage with faster cache-based access.

6.10 Data-Layer Limitations

The current data layer is suitable for a course prototype, but it has limitations:

Limitation	Future Improvement
Single PostgreSQL container	Use managed PostgreSQL or replication in production.
Single Redis container	Use Redis replication or cluster mode.
Prototype seed data	Add larger and more realistic datasets for performance testing.
Limited advanced analytics	Add reporting tables or warehouse design for analytics.
Basic cache invalidation	Improve cache invalidation rules for production use.

Despite these limitations, the data layer demonstrates relational database design, schema migration, seed data, constraints, indexes, and implemented Redis caching.

7. Batch Pipeline

7.1 Pipeline Overview

The project includes a daily restaurant settlement batch pipeline. The purpose of this pipeline is to calculate how much revenue each restaurant earned from completed orders and how much should be paid out after platform commission.

In a real food-delivery marketplace, restaurants need regular settlement reports. These reports summarize completed orders, total order value, platform commission, and final payout amount. The pipeline was designed to represent this business process.

The pipeline is not part of the interactive customer flow. Instead, it is a scheduled backend process that runs periodically, such as once per day.

7.2 Pipeline Goal

The main goal of the batch settlement pipeline is to process completed orders and generate settlement data for restaurants.

The pipeline answers the following questions:

Question	Description
Which orders were completed today?	The pipeline filters orders with final delivery status.
Which restaurant received each order?	Orders are grouped by restaurant.
What was the total revenue?	Order totals are summed for each restaurant.
What commission should the platform take?	A platform commission rate is applied.
What amount should be paid to each restaurant?	Final payout is calculated after commission.
Was the pipeline executed successfully?	Execution logs are written for audit and debugging.

7.3 Pipeline Input and Output

Input

The pipeline uses data from PostgreSQL.

Main input tables:

Table	Usage
orders	Provides order ID, restaurant ID, status, total price, and creation time.
order_items	Provides item-level details for orders.
restaurants	Provides restaurant information.
payments	Provides payment status and amount.
deliveries	Provides delivery completion status.

The most important input condition is that orders must be completed or delivered before being included in settlement.

Output

The pipeline output is a settlement summary for each restaurant.

Example output fields:

Field	Description
restaurant_id	Identifier of the restaurant.
settlement_date	Date of the settlement calculation.
completed_orders_count	Number of completed orders.
gross_revenue	Total value of completed orders.
platform_commission	Commission amount taken by the platform.
net_payout	Final payout amount for the restaurant.
execution_status	Whether the pipeline finished successfully.

7.4 BPMN-Style Pipeline Diagram

flowchart LR

Start((Start: Daily scheduled job)) --> ExtractOrders[Extract completed orders from PostgreSQL]

ExtractOrders --> ValidateOrders{Are completed orders available?}

ValidateOrders -- No --> NoData[Write no-data pipeline log]

NoData --> EndNoData((End))

ValidateOrders -- Yes --> GroupByRestaurant[Group orders by restaurant]

GroupByRestaurant --> CalculateTotals[Calculate total revenue, commission, and payout]

CalculateTotals --> GenerateSettlement[Generate settlement records]

GenerateSettlement --> SaveSettlement[Save settlement results to database]

SaveSettlement --> NotifyOwners[Prepare restaurant owner settlement summary]

NotifyOwners --> WriteLog[Write pipeline execution log]

WriteLog --> End((End: Settlement completed))

7.5 Pipeline Steps

Step	Description
1. Start scheduled job	The pipeline starts automatically based on a daily schedule.
2. Extract completed orders	The system selects orders that were completed or delivered.
3. Validate data	The pipeline checks whether there are orders to process.
4. Group by restaurant	Completed orders are grouped by restaurant ID.
5. Calculate totals	Gross revenue, commission, and net payout are calculated.
6. Generate settlement records	Settlement rows are prepared for storage.
7. Save results	Settlement results are saved to the database.
8. Write logs	Pipeline execution information is written to logs.
9. End job	The batch process finishes successfully.

7.6 Error Handling

The pipeline should handle several possible problems:

Error Case	Handling Strategy
No completed orders	Write a no-data log and finish successfully.
Database connection error	Log the error and mark the pipeline execution as failed.
Invalid order data	Skip invalid record or mark it for manual review.
Calculation error	Stop the pipeline, log details, and avoid saving incorrect results.
Partial failure	Use transactions to prevent incomplete settlement data.

For a production version, the pipeline should also include retry logic, alerting, and detailed audit logs.

7.7 Pipeline Limitations

The current pipeline is designed as a course-level prototype. It demonstrates the batch-processing concept, but it does not include all production-level requirements.

Limitation	Future Improvement
Daily schedule is conceptual	Add a real scheduler such as cron, Celery Beat, or Airflow.
Settlement table may be simplified	Add a dedicated settlement table with full audit fields.
Limited financial rules	Add tax, refunds, discounts, and restaurant-specific commission rules.
Basic logging	Add structured logs and alerting for failed jobs.
No external payment integration	Integrate real payment provider settlement reports.

Despite these limitations, the pipeline demonstrates the required batch-processing design and connects directly to the food-delivery business scenario.

8. From-Scratch Component: Token Bucket Rate Limiter

8.1 Component Overview

The from-scratch component implemented in this project is a Token Bucket Rate Limiter. Its purpose is to limit how frequently a client can send requests to the backend API. Rate limiting is important for protecting backend services from excessive traffic, accidental request loops, or simple abuse.

The Token Bucket algorithm was selected because it is simple, efficient, and commonly used in backend systems. It allows normal request bursts while still enforcing an average request rate over time.

8.2 Why This Component Was Chosen

A food-delivery marketplace can receive many repeated API requests, especially from frontend pages that load restaurant lists, menus, order status, and WebSocket-related data. Without rate limiting, a client could send too many requests and overload the backend.

The Token Bucket Rate Limiter was chosen because it supports:

Feature	Explanation
Burst handling	A client can send several requests quickly if tokens are available.
Average rate control	Tokens refill over time at a fixed rate.
Simple implementation	The algorithm can be implemented without external libraries.
Backend protection	It reduces the risk of excessive API requests.
Demonstration value	It is a clear from-scratch system component for the project.

8.3 Token Bucket Algorithm

The Token Bucket algorithm works with a bucket that stores tokens. Each client has a bucket. Every request consumes one token. Tokens are added back to the bucket over time. If the bucket has enough tokens, the request is allowed. If the bucket is empty, the request is rejected or delayed.

The basic logic is:

- 1. Create a bucket with a maximum capacity.
- 2. Refill tokens over time.
- 3. When a request arrives, check available tokens.
- 4. If at least one token exists, allow the request and remove one token.
- 5. If no token exists, reject the request with a rate-limit response.

8.4 Algorithm Parameters

Parameter	Description
Capacity	Maximum number of tokens that can be stored in the bucket.
Refill rate	Number of tokens added per time interval.
Current tokens	Number of available tokens at the current moment.
Last refill time	Last time the bucket was updated.
Client identifier	Value used to track each client separately.

Example:

```
capacity = 10 tokens
refill_rate = 1 token per second
request_cost = 1 token
```

This means a client can make a burst of up to 10 requests, and then continue making requests at an average rate of 1 request per second.

8.5 Backend Integration

The rate limiter is integrated into the FastAPI backend. It can be applied to API routes to check whether a request should be processed. The backend identifies the client, checks the token bucket, and either allows or blocks the request.

The simplified request flow is:

```
Incoming request
→ Rate limiter check
→ Token available?
    → Yes: continue to API handler
    → No: return rate limit response
```

This allows the backend to protect sensitive or frequently used endpoints from excessive traffic.

8.6 Pseudocode

```
function allow_request(client_id):
    bucket = get_bucket(client_id)

    now = current_time()
    elapsed = now - bucket.last_refill_time

    tokens_to_add = elapsed * refill_rate
    bucket.tokens = min(capacity, bucket.tokens + tokens_to_add)
    bucket.last_refill_time = now

    if bucket.tokens >= 1:
```

```
        bucket.tokens = bucket.tokens - 1
        return true

    return false
```

This pseudocode shows the core behavior of the rate limiter. The bucket is refilled based on elapsed time, and each allowed request consumes one token.

8.7 Component Benefits

Benefit	Description
Protects backend	Prevents too many requests from a single client.
Supports bursts	Allows short bursts of normal traffic.
Simple logic	Easy to understand and test.
Low overhead	Does not require heavy computation.
Reusable	Can be applied to multiple API endpoints.

8.8 Limitations

The current implementation is suitable for a course-level prototype, but it has limitations.

Limitation	Future Improvement
In-memory state	Store token buckets in Redis for distributed backend replicas.
Basic client identity	Use authenticated user ID or IP address more consistently.
Shared state across replicas	Move token bucket state to Redis for fully distributed rate limiting.
No dashboard for limiter events	Add metrics for allowed and blocked requests.
No advanced policies	Add endpoint-specific limits and user-role-based limits.

8.9 Production Improvements

For a production food-delivery system, the rate limiter should be improved by storing rate-limit state in Redis. This is important because the backend runs with multiple replicas. If rate-limit state is stored only in memory, each backend replica has its own independent bucket. Redis would allow all replicas to share the same rate-limit counters.

Recommended production improvements:

Improvement	Reason
Redis-backed buckets	Shared state across backend replicas.
Per-route limits	Configure different limits for public, authenticated, and admin endpoints.
Per-route limits	Different limits for login, order creation, and general browsing.
Metrics	Monitor blocked requests and suspicious clients.
Configuration	Store capacity and refill rates in environment variables.

The current implementation demonstrates the Token Bucket algorithm inside the backend and returns HTTP 429 Too Many Requests when the request limit is exceeded. Future improvements would make the limiter fully distributed by storing token bucket state in Redis and adding per-route configuration.

9. Infrastructure and Deployment

9.1 Infrastructure Overview

The infrastructure of the food-delivery marketplace is built with Docker Compose. The system is divided into multiple containers, each responsible for a specific part of the application. This design makes it possible to start the full stack with one command and keeps services isolated from each other.

9.0 DigitalOcean VPS Deployment

The project was deployed on a DigitalOcean VPS to demonstrate public access outside the local development environment. The repository was cloned on the server, environment variables were configured, and the full container stack was started with Docker Compose.

Public deployment URLs:

Service	URL
Frontend	http://138.68.88.189
Swagger / OpenAPI	http://138.68.88.189/docs
Restaurant API	http://138.68.88.189/api/restaurants
Grafana	http://138.68.88.189:3000/login
Prometheus	http://138.68.88.189:9090
Loki readiness endpoint	http://138.68.88.189:3100/ready

The VPS deployment ran the same main services as the local Docker Compose environment: Nginx gateway, two FastAPI backend replicas, React frontend, PostgreSQL, Redis, Prometheus, Grafana, Loki, and Promtail.

The infrastructure includes:

Service	Technology	Purpose
frontend	React + Vite	User interface for customers.
backend1	FastAPI	First backend replica for REST API and WebSocket logic.
backend2	FastAPI	Second backend replica for load balancing demonstration.
nginx	Nginx	API Gateway and single public entry point.
postgres	PostgreSQL	Main relational database.
redis	Redis	Cache and additional data store.
prometheus	Prometheus	Metrics collection.
grafana	Grafana	Metrics and logs dashboard.
loki	Loki	Log storage and querying.
promtail	Promtail	Log collection and forwarding to Loki.

9.2 Docker Compose Architecture

The application is started using Docker Compose. Docker Compose creates a shared network for the services and starts all required containers.

The main command used to start the system is:

```
docker compose up -d --build
```

The command used to stop the system is:

```
docker compose down
```

The container status can be checked with:

```
docker compose ps
```

During final testing, the following containers were running successfully:

```
food_backend1
food_backend2
food_frontend
food_gateway
food_grafana
food_loki
food_postgres
food_prometheus
food_promtail
food_redis
```

PostgreSQL and Redis were shown as healthy.

9.3 Docker Infrastructure Diagram

flowchart TB

```
Internet[User Browser] --> Nginx[Nginx API Gateway<br/>Public Port: 80]
```

```
Nginx --> Frontend[React Frontend Container<br/>Port: 5173]
```

```
Nginx --> Backend1[FastAPI Backend Replica 1<br/>Port: 8001 -> 8000]
```

```
Nginx --> Backend2[FastAPI Backend Replica 2<br/>Port: 8002 -> 8000]
```

```
Backend1 --> Postgres[(PostgreSQL Container<br/>Host Port: 5433<br/>Container Port: 5432)]
```

```
Backend2 --> Postgres
```

```
Backend1 --> Redis[(Redis Container<br/>Port: 6379)]
Backend2 --> Redis

Prometheus[Prometheus Container<br/>Port: 9090] --> Backend1
Prometheus --> Backend2

Grafana[Grafana Container<br/>Port: 3000] --> Prometheus
Grafana --> Loki

Promtail[Promtail Container] --> Loki[Loki Container<br/>Port: 3100]

subgraph DockerCompose[Docker Compose Network]
    Nginx
    Frontend
    Backend1
    Backend2
    Postgres
    Redis
    Prometheus
    Grafana
    Loki
    Promtail
end
```

9.4 Nginx API Gateway

Nginx is used as the API Gateway and the single public entry point of the system. The user accesses the application through:

`http://localhost`

The gateway routes traffic to the correct internal service.

Traffic Type	Route
Frontend traffic	Routed to the frontend container.
REST API traffic	Routed to FastAPI backend replicas.
WebSocket traffic	Routed to the backend WebSocket endpoint.

The gateway helps keep the public interface simple. Instead of exposing every service directly, the user interacts mainly with Nginx on port 80.

9.5 Backend Replicas and Load Balancing

The backend runs with two FastAPI replicas:

Replica	External Port	Internal Port
backend1	8001	8000
backend2	8002	8000

This demonstrates a load-balanced backend architecture. Nginx routes requests to backend replicas. This design improves scalability compared to a single backend container and demonstrates how a real system can distribute API traffic.

9.6 PostgreSQL and Redis

PostgreSQL is used as the main database. It stores persistent relational data such as users, restaurants, menu items, orders, payments, deliveries, and reviews.

Redis is used as an additional data store and cache. It can store frequently accessed data such as restaurant lists, menu data, order status, and rate limiter counters.

During local testing, PostgreSQL used host port 5433 because port 5432 was already occupied on the local machine. Inside Docker, PostgreSQL still used port 5432, so internal service communication was not changed.

Host port: 5433
Container port: 5432

9.7 Environment Configuration

The project uses environment variables to configure service connections and secrets. The `.env` file includes database, Redis, backend, frontend, gateway, and JWT configuration values.

Examples of environment variables:

```
POSTGRES_DB=food_delivery
POSTGRES_USER=food_user
POSTGRES_PASSWORD=food_password
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
REDIS_HOST=redis
REDIS_PORT=6379
DATABASE_URL=postgresql+psycopg2://food_user:food_password@postgres:5432/food_delivery
REDIS_URL=redis://redis:6379/0
BACKEND_PORT=8000
FRONTEND_PORT=5173
GATEWAY_PORT=80
```

The frontend also uses environment variables for API and WebSocket base URLs:

```
VITE_API_BASE_URL=http://localhost/api
VITE_WS_BASE_URL=ws://localhost/ws
```

9.8 Deployment Testing

The full system was tested through Docker Compose and Nginx Gateway. The final browser URL used for testing was:

`http://localhost`

The tested frontend flow was:

Restaurants

- View menu
- Add to cart
- Go to cart
- Create order
- Order Status
- WebSocket connected
- Move to next status

The final test confirmed that the frontend worked through the gateway and that the order status page displayed:

WebSocket: connected

9.9 Infrastructure Limitations

Limitation	Future Improvement
Local Docker Compose deployment	Deploy to a production VPS or cloud environment.
Single PostgreSQL container	Use managed PostgreSQL or replication.
Single Redis container	Use Redis replication or managed Redis.
Demo environment variables	Use production secret management.
No HTTPS in local setup	Add HTTPS through reverse proxy and certificates.
Basic gateway configuration	Add advanced routing, rate limits, and security headers.

The current infrastructure is sufficient for a course project demonstration and shows container orchestration, gateway routing, backend replicas, database, cache, and observability services.

10. Observability

10.1 Observability Overview

The project includes an observability stack to monitor the behavior of the food-delivery marketplace system. Observability is important because the application runs as multiple containers, including frontend, backend replicas, database, cache, gateway, and logging services.

The observability stack includes:

Tool	Purpose
Prometheus	Collects metrics from configured targets.
Grafana	Displays dashboards for metrics and logs.
Loki	Stores and queries logs.
Promtail	Collects container logs and forwards them to Loki.

This stack helps the team understand whether services are running, inspect logs, and capture screenshots for the final report.

10.2 Prometheus

Prometheus is used for metrics collection. It runs as a separate container and is available on:

```
http://localhost:9090
```

Prometheus can be used to check whether configured services are reachable and whether monitoring targets are active.

Prometheus is useful for:

Feature	Description
Target monitoring	Shows whether services are up or down.
Metric collection	Collects numeric time-series data.
Querying	Allows metric queries using PromQL.
Debugging	Helps identify service availability problems.

A screenshot of Prometheus targets was saved for the report.

10.3 Grafana

Grafana is used as the dashboard and visualization tool. It is available on:

```
http://localhost:3000
```

Grafana can display Prometheus metrics and Loki logs. It provides a visual interface for monitoring the system and presenting observability results in the final report.

Grafana is used to demonstrate:

Feature	Description
Dashboards	Visual panels for system metrics.
Logs view	Log search through Loki integration.
System visibility	A centralized view of application behavior.

A Grafana dashboard screenshot was saved for the report.

10.4 Loki and Promtail

Loki is used for log storage and querying, while Promtail collects logs from containers and forwards them to Loki.

The log flow is:

```
Docker containers
→ Promtail
→ Loki
→ Grafana
```

This allows the team to inspect backend, frontend, gateway, and infrastructure logs from Grafana.

Loki and Promtail are useful for:

Feature	Description
Centralized logs	Logs from multiple containers can be viewed in one place.
Debugging	Errors and request logs can be searched.
User action tracing	Logs can be checked after frontend actions such as order creation.
Report evidence	Log screenshots can be included in the final report.

10.5 Observability Screenshots

The project includes observability screenshots in the repository. These screenshots are used as evidence that the monitoring and logging stack was configured and tested.

Screenshots include:

Screenshot	Purpose
prometheus-targets.png	Shows Prometheus target status.
grafana-dashboard.png	Shows Grafana dashboard.
loki-logs.png	Shows logs available through Loki/Grafana.
docker-containers.png	Shows running Docker containers.
gateway-frontend.png	Shows frontend access through Nginx gateway.

Frontend screenshots were also saved separately:

Screenshot	Purpose
01-restaurant-list.png	Shows restaurant list page.
02-menu-page.png	Shows menu page.
03-cart-page.png	Shows cart page.
04-order-status-page.png	Shows order status page.
05-websocket-connected.png	Shows WebSocket connected state.
06-status-updated.png	Shows status update after action.

10.6 Observability Testing

The observability stack was tested together with the full Docker Compose environment. The following command was used to verify running containers:

```
docker compose ps
```

The system showed the main services running:

```
food_backend1
food_backend2
food_frontend
food_gateway
food_grafana
food_loki
food_postgres
food_prometheus
food_promtail
food_redis
```


PostgreSQL and Redis were healthy. The frontend was accessible through:

```
http://localhost
```

Prometheus, Grafana, Loki, and Promtail containers were also running.

10.7 Observability Limitations

The current observability setup is suitable for a course-level prototype, but it has limitations.

Limitation	Future Improvement
Basic dashboards	Add more detailed dashboards for API latency, error rates, and request volume.
Limited application metrics	Add custom FastAPI metrics for important endpoints.
Logs only at container level	Add structured JSON logs with request IDs.
No alerting	Add Grafana or Prometheus alerts for service failures.
Limited tracing	Add distributed tracing for full request lifecycle tracking.

Despite these limitations, the observability stack demonstrates service monitoring, log collection, dashboard usage, and report evidence.

11. Frontend Implementation

11.1 Frontend Overview

The frontend application was implemented using React and Vite. It provides a simple customer-facing interface for the food-delivery marketplace. The frontend focuses on demonstrating the main user flow rather than complex visual design.

The implemented frontend flow is:

```
Restaurant list
→ Menu page
→ Cart page
→ Order creation
→ Order status page
→ Live WebSocket update
```

The frontend was first developed with mock data because the backend was not fully ready at the beginning. After the backend mock API was merged, the frontend was connected to the real backend endpoints while keeping mock data as a fallback mechanism.

11.2 Frontend Pages

The frontend includes the following pages:

Page	Purpose
Restaurant List Page	Displays available restaurants.
Menu Page	Displays menu items for the selected restaurant.
Cart Page	Shows selected food items and total price.
Order Status Page	Shows order status and WebSocket connection state.

These pages demonstrate the complete customer journey from browsing restaurants to tracking an order.

11.3 Frontend API Client

The frontend uses an API client to communicate with the backend. The API client centralizes REST API calls and provides fallback mock data if the backend is unavailable.

The API client supports:

Function	Backend Endpoint
getRestaurants	GET /api/restaurants
getMenuByRestaurantId	GET /api/restaurants/{restaurant_id}/menu
createOrder	POST /api/orders
getOrderById	GET /api/orders/{order_id}
updateOrderStatus	PATCH /api/orders/{order_id}/status

The frontend uses environment variables to configure backend connection URLs:

```
VITE_API_BASE_URL=http://localhost/api
VITE_WS_BASE_URL=ws://localhost/ws
```

This configuration allows the frontend to work through the Nginx gateway in Docker Compose.

11.4 Mock Data Fallback

Mock data was kept as a fallback mechanism. This was useful because frontend development started before all backend endpoints were fully ready.

The fallback data includes:

Mock Data	Purpose
restaurants	Allows the restaurant list page to work without backend.
menuItems	Allows the menu page to show food items without backend.
fakeOrder	Allows the order status page to work without backend.

This approach allowed frontend development and testing to continue independently. After backend integration, the frontend could use real API responses while still remaining stable if one endpoint temporarily failed.

11.5 Cart and Order Creation Flow

The cart page allows users to review selected menu items before creating an order. Items are stored in local frontend state and browser storage during the prototype flow. When the user clicks “Create order”, the frontend sends an order creation request to the backend.

The order creation payload includes:

```
{
  "restaurant_id": 1,
  "customer_id": 1,
  "delivery_address": "Demo address, Tashkent",
```

```
"total_price": 19.98,  
"items": [  
  {  
    "menu_item_id": 101,  
    "quantity": 1  
  }  
]
```

After the backend returns the created order ID, the frontend redirects the user to the order status page.

11.6 WebSocket Integration

The order status page includes WebSocket integration for live order status updates. When the page opens, the frontend connects to:

```
ws://localhost/ws/orders/{order_id}
```

If the connection is successful, the page displays:

```
WebSocket: connected
```

The backend can then send status update messages to the frontend. The frontend updates the visible status without refreshing the page.

The supported order status lifecycle is:

```
created → confirmed → preparing → picked_up → delivered
```

This feature demonstrates the real-time requirement of the system.

11.7 Frontend Screenshots

Frontend screenshots were saved in the repository under:

```
docs/screenshots/frontend/
```

The saved screenshots include:

Screenshot	Description
01-restaurant-list.png	Restaurant list page.
02-menu-page.png	Menu page with menu items.
03-cart-page.png	Cart page before order creation.
04-order-status-page.png	Order status page.
05-websocket-connected.png	Order status page showing WebSocket connected.
06-status-updated.png	Order status page after status update.

The most important screenshot for the WebSocket requirement is:

```
05-websocket-connected.png
```

It shows that the order status page successfully connected to the backend WebSocket endpoint.

11.8 Frontend Testing

The frontend was tested in two modes:

Mode	Purpose
Local Vite development	Tested React pages with <code>npm run dev</code> .
Docker Compose through Nginx	Tested the full system through <code>http://localhost</code> .

The final tested flow was:

```
Open http://localhost
→ Open restaurant list
→ View menu
→ Add item to cart
→ Go to cart
→ Create order
→ Open order status page
→ Confirm WebSocket connected
→ Move order to next status
```

The final Docker Compose test confirmed that the frontend worked through the gateway and that WebSocket status was connected.

11.9 Frontend Limitations

Limitation	Future Improvement
Simple UI design	Improve visual design and responsiveness.
Basic cart logic	Store cart data in backend or authenticated user session.
Demo customer ID	Use real authenticated user identity.
Limited error handling	Add user-friendly error messages and retry behavior.
Basic WebSocket handling	Add automatic reconnect and connection recovery.
Mock fallback still present	Keep only controlled fallback behavior in production.

Despite these limitations, the frontend demonstrates the required marketplace flow and real-time order status updates.

12. Testing and Known Limitations

12.1 Testing Overview

The project was tested through local development and Docker Compose integration. The goal of testing was to confirm that the main system components work together and that the food-delivery user flow can be demonstrated successfully.

The main tested areas were:

Area	Tested Functionality
Frontend	Restaurant list, menu page, cart, order creation, order status page.
Backend API	Restaurant endpoints, menu endpoint, order creation, order retrieval, order status update.
WebSocket	Live order status connection and update display.
Database	PostgreSQL container startup, migrations, seed data, and query support.
Redis	Redis container startup and cache design readiness.
Docker Compose	Multi-container startup and service networking.
Nginx Gateway	Frontend, API, and WebSocket access through <code>http://localhost</code> .
Observability	Prometheus, Grafana, Loki, and Promtail containers.

12.2 Frontend Testing

The frontend was tested first in local Vite development mode and later through the full Docker Compose system.

Local frontend command:

```
npm run dev
```

Docker Compose access URL:

```
http://localhost
```

The final frontend test flow was:

```
Open http://localhost
→ View restaurant list
```

- Open restaurant menu
- Add item to cart
- Open cart
- Create order
- Open order status page
- Confirm WebSocket connected
- Move order to next status

The final result confirmed that the frontend was working through the gateway and that the WebSocket state showed:

WebSocket: connected

12.3 Backend API Testing

The backend API was tested through the frontend flow and Swagger documentation.

Swagger URL:

`http://127.0.0.1:8000/docs`

The important backend endpoints were:

Method	Endpoint	Test Result
GET	/api/restaurants	Restaurant list loaded successfully.
GET	/api/restaurants/{restaurant_id}/menu	Menu items loaded successfully.
POST	/api/orders	Order creation worked.
GET	/api/orders/{order_id}	Order status page loaded order data.
PATCH	/api/orders/{order_id}/status	Order status update worked.
WS	/ws/orders/{order_id}	WebSocket connection worked.

12.4 Docker Compose Testing

The full stack was tested using Docker Compose.

Commands used:

```
docker compose down
docker compose up -d --build
docker compose ps
```

The final Docker Compose test showed that the required containers were running:

```
food_backend1
food_backend2
food_frontend
food_gateway
food_grafana
food_loki
food_postgres
food_prometheus
```

```
food_promtail  
food_redis
```

PostgreSQL and Redis were shown as healthy. The frontend was accessible through Nginx on:

```
http://localhost
```

12.5 WebSocket Testing

WebSocket testing was important because it demonstrates the real-time requirement of the project.

The tested WebSocket endpoint pattern was:

```
/ws/orders/{order_id}
```

The frontend displayed the connection state on the order status page. The final screenshot showed:

```
WebSocket: connected
```

After pressing the status update button, the order status changed from one state to the next. This demonstrated live order status update behavior.

12.6 Screenshot Evidence

The following screenshots were collected for the report:

Screenshot	Evidence
01-restaurant-list.png	Restaurant list page works.
02-menu-page.png	Menu page works.
03-cart-page.png	Cart page works.
04-order-status-page.png	Order status page works.
05-websocket-connected.png	WebSocket connected state works.
06-status-updated.png	Status update works.
docker-containers.png	Docker containers are running.
gateway-frontend.png	Frontend is available through gateway.
prometheus-targets.png	Prometheus target page.
grafana-dashboard.png	Grafana dashboard.
loki-logs.png	Loki logs.

12.7 Known Limitations

The current system is a course-level prototype and has several limitations.

Limitation	Explanation
Simple frontend UI	The frontend focuses on demonstrating functionality rather than advanced design.
Demo user data	Some flows use demo customer data instead of full authentication.
Local storage cart	Cart data is handled on the frontend for the prototype.
Single PostgreSQL instance	The database is not replicated.
Single Redis instance	Redis is not configured as a cluster.
Basic deployment	The system was mainly tested through local Docker Compose.
Limited production security	HTTPS, secure secret management, and full role-based access should be added for production.
Basic observability	Dashboards and logs are sufficient for demonstration but not production-grade.
Prototype batch pipeline	The settlement pipeline is designed conceptually and should be extended for production scheduling.

12.8 Future Improvements

Future improvements could include:

Improvement	Description
Full authentication	Add complete login, registration, and role-based access control.
Better cart persistence	Store cart data on the backend for authenticated users.
Production deployment	Deploy the system to a VPS or cloud provider with HTTPS.
Advanced observability	Add custom metrics, alerts, and distributed tracing.
Improved WebSocket reliability	Add reconnect logic and persistent event delivery.
Payment provider integration	Integrate a real payment gateway.
Courier app features	Add courier assignment and delivery map features.
Admin dashboard	Add admin tools for monitoring orders and restaurants.
Advanced batch pipeline	Add scheduled execution, settlement table, and financial audit records.

12.9 Testing Conclusion

The final testing confirmed that the main project requirements were demonstrated successfully. The application runs through Docker Compose, the frontend is accessible through Nginx Gateway, the REST API supports the main business flow, and WebSocket live order tracking works on the order status page.

The project is not production-ready, but it successfully demonstrates a full-stack database application with frontend, backend, database, cache, WebSocket, Docker infrastructure, observability, diagrams, screenshots, and documentation.

13. Team Contribution Table

The project was completed as a team project. Each team member was responsible for a specific part of the system. The table below summarizes the main responsibilities and completed contributions.

Team Member	Role	Main Responsibilities	Completed Contributions
Maxsetov Salamat	Frontend + Report	React frontend, user flow, WebSocket demonstration, screenshots, report sections	Implemented restaurant list page, menu page, cart page, order creation flow, and order status page. Connected frontend to REST API and WebSocket endpoint. Captured frontend screenshots for the report. Prepared frontend implementation text and diagrams.
Maxsetov Nurmukhamad	Backend Core	FastAPI backend, REST API, WebSocket, authentication, Redis cache, and rate limiting	Implemented backend API endpoints, JWT authentication, WebSocket backend logic, order status updates, Redis cache integration, and Token Bucket Rate Limiter.
Maxsetov Salamat	Database + Cache	PostgreSQL schema, Alembic migrations, seed data, indexes, and Redis cache design	Designed and implemented database schema, relationships, constraints, Alembic migrations, seed data, performance indexes, query optimization results, and Redis cache design.
Mirjalilov Mirkamol	Docker Infrastructure + Observability	Docker Compose, Nginx gateway, backend replicas, observability stack, and VPS deployment	Configured Docker Compose, Nginx API Gateway, backend replicas, PostgreSQL, Redis, frontend container, Prometheus, Grafana, Loki, Promtail, and DigitalOcean VPS deployment.

14. References

1. FastAPI Documentation. Used for backend REST API and WebSocket implementation.
2. React Documentation. Used for frontend component structure and state management.
3. Vite Documentation. Used for frontend development server and build setup.
4. PostgreSQL Documentation. Used for relational database design and SQL constraints.
5. SQLAlchemy Documentation. Used for ORM models and backend database access.
6. Alembic Documentation. Used for database migrations.
7. Redis Documentation. Used for cache and additional data store design.
8. Docker Documentation. Used for containerization.
9. Docker Compose Documentation. Used for multi-container orchestration.

- 10. Nginx Documentation. Used for API Gateway and reverse proxy configuration.
- 11. Prometheus Documentation. Used for metrics collection.
- 12. Grafana Documentation. Used for dashboards and visualization.
- 13. Loki and Promtail Documentation. Used for log collection and querying.
- 14. Mermaid Documentation. Used for ER, architecture, infrastructure, and BPMN-style diagrams.

Appendix A. Screenshots

The following screenshots were collected and stored in the project repository.

Screenshot	Location
Restaurant list page	docs/screenshots/frontend/01-restaurant-list.png
Menu page	docs/screenshots/frontend/02-menu-page.png
Cart page	docs/screenshots/frontend/03-cart-page.png
Order status page	docs/screenshots/frontend/04-order-status-page.png
WebSocket connected	docs/screenshots/frontend/05-websocket-connected.png
Status updated	docs/screenshots/frontend/06-status-updated.png
Docker containers	docs/screenshots/docker-containers.png
Gateway frontend	docs/screenshots/gateway-frontend.png
Grafana dashboard	docs/screenshots/grafana-dashboard.png
Prometheus targets	docs/screenshots/prometheus-targets.png
Loki logs	docs/screenshots/loki-logs.png

Appendix B. Diagrams

The following diagrams were prepared for the project report.

Diagram	Location
System architecture diagram	docs/diagrams/system_architecture.md
Docker infrastructure diagram	docs/diagrams/docker_infrastructure.md
BPMN-style batch settlement pipeline diagram	docs/diagrams/batch_settlement_bpmn.md
ER diagram	docs/er_diagram.md

Final Conclusion

The food-delivery marketplace project demonstrates a complete database application with frontend, backend, database, cache, real-time communication, batch pipeline design, infrastructure, observability, and documentation. The system supports the main business scenario of browsing restaurants, viewing menus, creating orders, and tracking order status updates through WebSocket.

The final implementation was tested through Docker Compose and Nginx Gateway. The frontend was accessible through `http://localhost`, the backend API supported the main user flow, and the order status page successfully showed `WebSocket: connected`. The project also includes screenshots, diagrams, database documentation, and a final report structure suitable for submission.